

---

# Introduction to OpenShift Virtualization

Like other Red Hat virtualization products, OpenShift Virtualization uses *Kernel-based Virtual Machine* (KVM) technology as the hypervisor and the libvirt libraries for defining the virtual machines. KVM and libvirt are run in a container that can be deployed in Kubernetes and OpenShift platforms along with containerized applications. The *Quick EMUlator* (QEMU) guest agent passes information about a VM to the host.

Red Hat OpenShift is a set of modular components and services that are built on top of Kubernetes container infrastructure. The Red Hat OpenShift software product is derived from the Kubernetes open source project. Although Red Hat OpenShift and Kubernetes are container orchestration software, Red Hat OpenShift is packaged as a downstream enterprise open source platform.

## Key Points

### OpenShift Virtualization uses KVM & libvirt

- *What it means:* Like other Red Hat products (e.g., RHEL, RHV), OpenShift Virtualization uses **Kernel-based Virtual Machine (KVM)** technology to create and run virtual machines.
- The **libvirt** library is used to define and manage these VMs.
- This virtualization stack runs inside a **container**, so it can be deployed just like any other containerized application in OpenShift.

### QEMU (Quick EMUlator) guest agent

- *What it means:* The **QEMU** software is used to emulate hardware for the virtual machines.
- A QEMU **guest agent** runs inside the VM to pass information back to the OpenShift host, like IP address, hostname, and status.

## Integration with Kubernetes/OpenShift

- *What it means:* OpenShift Virtualization is **not a separate hypervisor product** — it is built **on top of Kubernetes/OpenShift**.
- OpenShift is a **container orchestration platform**. Virtualization is just an **add-on** service running as containers and operators in OpenShift.

## Downstream Enterprise Product

- *What it means:* OpenShift is based on the open-source Kubernetes project but packaged by Red Hat as an **enterprise-grade product** with support, updates, and tested integrations.

## Example

### Scenario:

Imagine you have a Kubernetes cluster running OpenShift. You want to run a legacy application that only works on a virtual machine — for example, an old Windows Server that runs a specific .NET app.

### With OpenShift Virtualization:

- You deploy a **VirtualMachine (VM) object** using KubeVirt (the upstream project behind OpenShift Virtualization).
- Under the hood, KVM + libvirt spin up the VM inside a container pod.
- QEMU emulates the hardware for the VM.
- The VM runs on the same OpenShift cluster as your containerized workloads — it can share the same storage, networking, and security policies.
- You manage it just like a Kubernetes pod: `oc get vm`, `oc start vm`, etc.

## Why it's useful

- You can run **containers and VMs side-by-side** on the same OpenShift cluster.

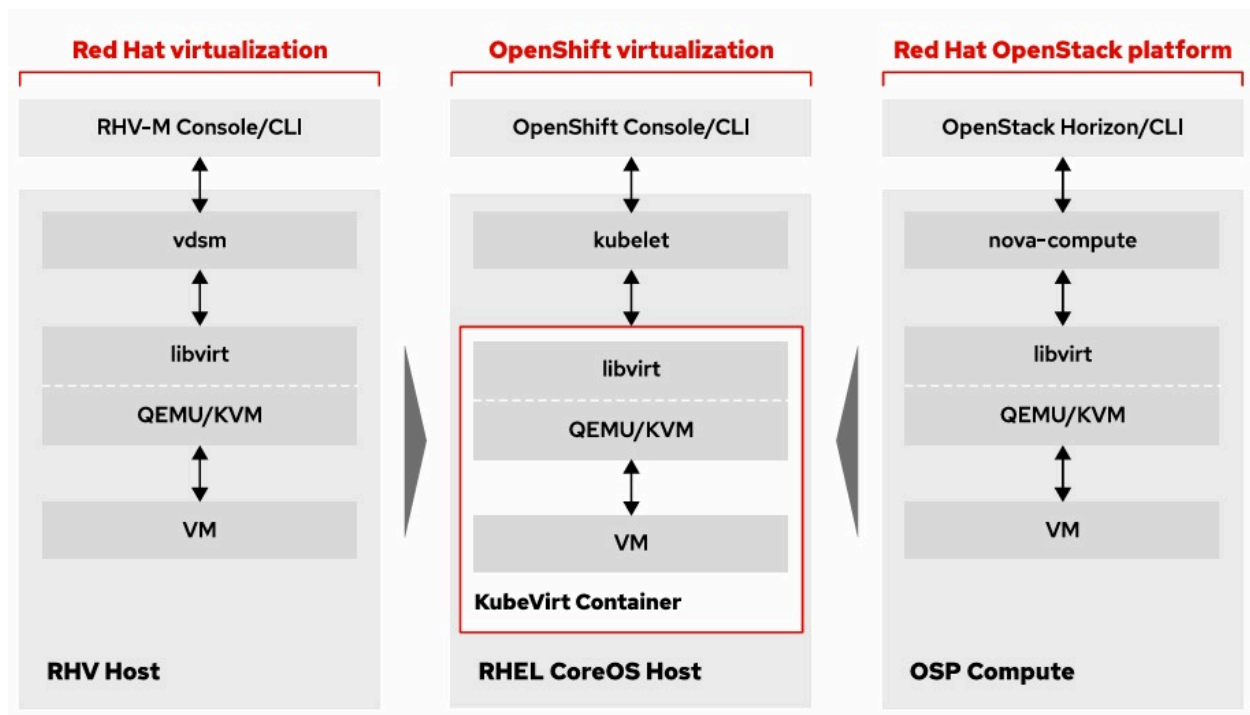
- It helps migrate traditional workloads to the cloud-native world without rewriting apps immediately.
- Everything is managed consistently using Kubernetes APIs.

\*\*\*\*\*

## Containerizing KVM

The OpenShift Virtualization operator offers features for enterprise-grade virtualization, such as live migration, high availability, system scheduling, power management, image management, snapshots, thin provisioning, and monitoring.

Red Hat OpenShift, with the Virtualization operator installed, manages both containers and virtual machines from a common interface.



# Key Technologies & Full Forms

| Abbreviation    | Full Form                          | What it Does                                                               |
|-----------------|------------------------------------|----------------------------------------------------------------------------|
| <b>KVM</b>      | Kernel-based Virtual Machine       | Linux-based hypervisor technology for running VMs.                         |
| <b>QEMU</b>     | Quick EMUlator                     | Open source emulator that provides hardware emulation for VMs.             |
| <b>libvirt</b>  | — (Library Virtualization)         | API and management tool to define and control VMs (manages KVM/QEMU).      |
| <b>RHV</b>      | Red Hat Virtualization             | Red Hat's standalone virtualization platform based on KVM.                 |
| <b>RHV-M</b>    | Red Hat Virtualization Manager     | Management console for RHV.                                                |
| <b>VDSM</b>     | Virtual Desktop and Server Manager | Daemon in RHV that manages virtualization tasks on the host.               |
| <b>OSP</b>      | OpenStack Platform                 | Red Hat's version of the OpenStack cloud IaaS platform.                    |
| <b>Nova</b>     | — (Nova Compute Service)           | OpenStack component for provisioning and managing compute instances (VMs). |
| <b>KubeVirt</b> | — (Kubernetes Virtualization)      | Open source project to run VMs in Kubernetes.                              |

|                    |                                 |                                                                                                            |
|--------------------|---------------------------------|------------------------------------------------------------------------------------------------------------|
| <b>CLI</b>         | Command Line Interface          | Text-based tool to manage and control systems.                                                             |
| <b>VM</b>          | Virtual Machine                 | Emulated computer system running an OS and applications.                                                   |
| <b>RHEL CoreOS</b> | Red Hat Enterprise Linux CoreOS | Immutable, container-optimized OS used as the node OS in OpenShift.                                        |
| <b>kubelet</b>     | — (Kubernetes Node Agent)       | Primary agent that runs on each node in a Kubernetes cluster to manage containers (and VMs with KubeVirt). |

## 1. Red Hat Virtualization (RHV)

- **RHVM** (Red Hat Virtualization Manager) is the main console (web UI + CLI) used to manage virtual machines.
- It talks to **VDSM (Virtual Desktop and Server Manager)** on the RHV Host.
- **VDSM** controls **libvirt**.
- **libvirt** manages **QEMU** (Quick EMUlator) and **KVM** (Kernel-based Virtual Machine) to run the actual VMs (Virtual Machines).

### Example:

You have a physical RHV cluster. You run 20 Windows Server VMs for your data center workloads. You manage them via RHVM.

## 2. OpenShift Virtualization

- Uses **OpenShift Console** (web UI) or **CLI (Command Line Interface)**.
- Instead of VDSM, it uses **kubelet** — the Kubernetes node agent.
- The **libvirt + QEMU/KVM** stack runs **inside a KubeVirt Container**.

- This containerized virtualization stack runs alongside other containers on **RHEL CoreOS (Red Hat Enterprise Linux CoreOS)** hosts.
- You create VMs using **KubeVirt**, which exposes VMs as Kubernetes resources.

**Example:**

Same as before — you need to run an old Oracle DB VM, but now inside your OpenShift cluster. You deploy a **VirtualMachine CRD** (Custom Resource Definition) managed by KubeVirt.

### 3. Red Hat OpenStack Platform (OSP)

- You manage VMs with **OpenStack Horizon** (web UI) or the **CLI**.
- The **nova-compute** service talks to **libvirt**, which controls **QEMU/KVM** to run VMs.
- This is the standard OpenStack compute architecture.

**Example:**

You use OpenStack to provide Infrastructure-as-a-Service (IaaS). Different teams deploy and manage VMs for different tenants using Horizon.

### Why OpenShift Virtualization is Unique

- **KVM (Kernel-based Virtual Machine)** and **QEMU (Quick EMUlator)** are not separate services — they are containerized and orchestrated by Kubernetes via **KubeVirt**.

### So, in OpenShift Virtualization:

- VMs are scheduled like Pods.
- You use **kubelet** instead of **vdsm** or **nova-compute**.
- You get full Kubernetes benefits: scaling, self-healing, monitoring.
- Everything is integrated in **RHEL CoreOS**.

| Layer      | RHV (Red Hat Virtualization)              | OpenShift Virtualization                 | OpenStack (OSP)         |
|------------|-------------------------------------------|------------------------------------------|-------------------------|
| Management | RHVM (Red Hat Virtualization Manager)     | OpenShift Console + CLI                  | OpenStack Horizon + CLI |
| Node Agent | VDSM (Virtual Desktop and Server Manager) | kubelet                                  | nova-compute            |
| Hypervisor | libvirt + QEMU/KVM                        | libvirt + QEMU/KVM in KubeVirt Container | libvirt + QEMU/KVM      |
| Guest      | VM                                        | VM                                       | VM                      |

## Full Example Workflow

### Example:

You want to run a legacy Windows Server 2012 VM on OpenShift Virtualization.

### How it works:

1. Create a **VirtualMachine** manifest (YAML) using **KubeVirt**.
2. **OpenShift Console** talks to **kubelet**.
3. **kubelet** schedules a pod on a **RHEL CoreOS** host.
4. That pod runs **libvirt + QEMU/KVM** to launch the **VM** inside the pod.
5. You access the VM via **VNC/SPICE** or assign a service IP — just like a container service.

## Full Benefit

- Same OpenShift cluster = **Containers + VMs**.
- Shared networking, storage (like **CSI, PVC, CNV**).
- Unified monitoring and RBAC.
- Enterprise features: **Live migration, snapshots, high availability**, etc.

\*\*\*\*\*

## Virtualization Meets Containerization

This course explores the features of OpenShift Virtualization that provide the management and orchestration layer for virtual machine workloads within a Red Hat OpenShift cluster.

With OpenShift Virtualization, you can deliver the resilience and ease of managing both VM-based and containerized applications from a single interface and infrastructure. This capability eliminates the need for a separate and potentially expensive virtualization management solution and creates a valuable opportunity for businesses that are undergoing container adoption and still maintain a traditional deployment of virtual machines.

## Benefits of OpenShift Virtualization

Adding OpenShift Virtualization to a Red Hat OpenShift cluster enables deploying and managing virtualized workloads with the same hardware, environment, and tool set that the containerized applications use.

Managing virtual machines alongside containerized deployments in a single cluster provides both administrative and economical advantages. Using OpenShift



Virtualization unifies the management interfaces that system administrators use, to improve application reliability and reduce operational overhead.

The following operators and components are integrated for the OpenShift Virtualization operator:

## OpenShift Virtualization Operator

OpenShift Virtualization is an open-source technology to enable running VMs in Red Hat OpenShift with the OpenShift Virtualization operator. It is based on the upstream **KubeVirt** project, and runs and manages traditional VMs as if they were containers.

## Key Idea: Virtualization Meets Containerization

What it says:

- OpenShift Virtualization lets you run **virtual machines (VMs)** and **containerized workloads** together on the same **Red Hat OpenShift cluster**.
- The **OpenShift Virtualization Operator** provides the **management and orchestration layer** for VMs — just like Kubernetes does for containers.
- This removes the need for a **separate virtualization platform** (like RHV or VMware vSphere) when you already run OpenShift.

## Benefits Explained

### 1. Single Infrastructure

- Use the same hardware and network for **containers and VMs**.
- Example: No separate RHV or OpenStack cluster needed.

### 2. Unified Management

- Use the same **OpenShift Console & CLI** to manage both Pods and VMs.

- Example: `oc get pods` and `oc get vm` — same tool, same cluster.

### 3. Operational Efficiency

- Admins use the same automation, **RBAC (Role-Based Access Control)**, monitoring, and storage for both types of workloads.
- Example: Reuse the same **Persistent Volume Claims (PVCs)** for VMs and containers.

### 4. Cost Saving & Simplicity

- No extra hypervisor layer to maintain.
- Example: You don't pay or maintain a separate RHV-M or OpenStack Horizon.

## Key Component: OpenShift Virtualization Operator

- An Operator in Kubernetes is a method for automating application lifecycle.
- The OpenShift Virtualization Operator installs and manages KubeVirt (the upstream project).
- KubeVirt containerizes libvirt and QEMU/KVM so that VMs run as pods.
- The Operator ensures the KubeVirt components are healthy, upgraded, and integrated.

## Example to Illustrate

### Scenario:

Your company is moving to containers but still has some legacy apps:

- An old **Windows Server 2012** with a licensing server that can't be containerized.
- A **RHEL 7 Oracle DB** that's not container-ready.

But your new apps are microservices deployed in **containers on OpenShift**.

## Without OpenShift Virtualization

- You would keep separate clusters:
  - VMware vSphere or RHV for VMs.
  - OpenShift for containers.
- Different hardware pools.
- Separate monitoring and backups.
- Different admin teams.

## With OpenShift Virtualization

1. Install the **OpenShift Virtualization Operator** on your existing OpenShift cluster.
2. The Operator deploys **KubeVirt**:
  - KubeVirt runs **libvirt + QEMU/KVM** inside pods.
3. You define a **VirtualMachine (VM)** manifest in YAML.

```
apiVersion: kubevirt.io/v1
kind: VirtualMachine
metadata:
  name: windows2012-lic-server
spec:
  running: true
  template:
    spec:
      domain:
        cpu:
          cores: 2
        devices:
          disks:
            - disk:
                bus: virtio
                name: rootdisk
      resources:
        requests:
          memory: 4Gi
      volumes:
        - name: rootdisk
          containerDisk:
            image: KubeVirt/win2012-container-disk-demo:latest
```

4. OpenShift schedules this **VM like any pod**.

- Runs QEMU/KVM inside the KubeVirt container.
- VM gets a Pod IP, connects to cluster networking.

5. You now have microservices and VMs managed side by side:

- Same OpenShift Console to monitor pods and VMs.
- Same storage backend (CSI PVCs).
- Same backup tools and logging.

## Result

- Your legacy Windows licensing server stays up.
- New apps run as pods.
- You have **one cluster, one toolset, one admin team**.
- You deliver faster with **lower operational overhead**.

| Term     | Full Form                    |
|----------|------------------------------|
| VM       | Virtual Machine              |
| RHV      | Red Hat Virtualization       |
| KVM      | Kernel-based Virtual Machine |
| QEMU     | Quick EMUlator               |
| KubeVirt | Kubernetes Virtualization    |
| PVC      | Persistent Volume Claim      |
| CSI      | Container Storage Interface  |
| RBAC     | Role-Based Access Control    |

## Bottom Line

Virtualization Meets Containerization means:

Run **legacy workloads (VMs)** and **modern apps (containers)** together — fully managed by **OpenShift** with the **OpenShift Virtualization Operator**.

## HyperConverged Operator (HCO)

The HCO deploys and manages virtualization components and supplemental operators, such as the **Containerized Data Importer (CDI)**, for OpenShift Virtualization.

---

## Hostpath Provisioner Operator (HPP)

The HPP operator is a local storage provisioner for virtual machines. Local storage or mounted volumes on nodes can be provisioned to **Virtual Machine Instances (VMIs)**.

---

## QEMU agent

The **Quick EMUlator (QEMU)** guest agent is a daemon that runs inside the VM and passes information to the host about the VM, users, file systems, and secondary networks.

---

## Kernel-based Virtual Machine (KVM)

KVM is an open source virtualization technology that is built into the Linux kernel that enables a host machine to run multiple, isolated virtual environments, **guest VMs**.

---

## Containerized Data Importer (CDI)

An upstream Kubernetes component that delivers a hardware-independent management layer for storage volumes that attach to deployed instances.

## Virtual machine consoles

Connect to VMs by using command-line tools, the web console, and common protocols, such as **Remote Desktop Protocol (RDP)** for Windows VMs, and **SSH**.

---

## Stand-alone Virtual Machine Instances (VMIs)

Use the command line and the web console to manage stand-alone VMIs that were created independently from the OpenShift cluster, such as VMs that were created by using scripts, with automation, or with other command-line methods.

## Key Operators & Components Explained

1. **HyperConverged Operator (HCO)** Full Form: HCO — **HyperConverged Operator**
  - **What it does:**
    - It deploys and manages all the core pieces of OpenShift Virtualization.
    - It bundles other important operators like the **Containerized Data Importer (CDI)**.
    - Think of it as the "master operator" for virtualization.
  - **Why it matters:**
    - You don't install each component manually — HCO takes care of orchestration.
  - **Example:** When you enable virtualization in OpenShift, you actually install the HCO. It then installs KubeVirt, CDI, and other helpers automatically.

## 2. Hostpath Provisioner Operator (HPP)

Full Form: HPP — **Hostpath Provisioner**

- **What it does:**
  - Provides **local storage** for Virtual Machines.
  - Lets you use the local disks of nodes for storing VM data.
- **Why it matters:**
  - Some test/dev environments need local disks instead of shared storage.
  - Works well when you don't have an external storage system.

**Example:** You spin up a VM for testing and attach a local disk as its root volume via HPP.

## 3. QEMU Agent

Full Form: QEMU — **Quick EMUlator**

- **What it does:**
  - A small program that runs inside each VM.
  - Passes information about the VM's status, users, file systems, and network back to the host.
- **Why it matters:**
  - Enables advanced features like live migration, dynamic IP detection, and status monitoring.

**Example:** The QEMU agent inside a Windows VM reports its IP back to OpenShift, so the console shows it correctly.

## 4. Kernel-based Virtual Machine (KVM)

Full Form: KVM — **Kernel-based Virtual Machine**

- **What it does:**
  - A Linux kernel module that turns the Linux server into a hypervisor.
  - Enables the host to run multiple, isolated virtual machines (guests).

**Example:** When you run a VM in OpenShift Virtualization, KVM actually handles the virtualization on the physical node.

## 5. Containerized Data Importer (CDI)

Full Form: CDI — **Containerized Data Importer**

- **What it does:**
  - A Kubernetes-native tool to import disk images into PVCs (Persistent Volume Claims).
  - Lets you upload VM images (like `.qcow2` or `.iso`).
- **Why it matters:**
  - Simplifies onboarding VMs from other platforms (VMware, RHV, bare metal).

**Example:** You upload a CentOS image with CDI, which writes the disk to your OpenShift storage. Then you create a VM using that disk.

## 6. Virtual Machine Consoles

- **What it does:**
  - Let you connect to your VMs.
  - Supports CLI access, web-based consoles, or standard protocols like **RDP (Remote Desktop Protocol)** for Windows and **SSH** for Linux.
- **Why it matters:**
  - You manage VMs directly, just like with RHV or VMware.

**Example:** You open the OpenShift Console, click your VM, launch the VNC console, and log in.

## 7. Stand-alone Virtual Machine Instances (VMIs)

Full Form: VMI — **Virtual Machine Instance**

- **What it does:**
  - A **running** virtual machine.
  - Unlike a **VirtualMachine** object (which is a template and stateful), a VMI can be created directly for testing or automation.
- **Why it matters:**
  - Good for quickly spinning up test VMs without full config.

**Example:** You run a one-time test VM using a `kubectl run` or script, which creates a **VMI** instead of a persistent **VirtualMachine**.

### How it all fits together

Putting it together with a mini-scenario:



- **Example:**

1. You install **HCO (HyperConverged Operator)** in your OpenShift cluster.
  - HCO deploys **KubeVirt**, **CDI**, and **HPP** automatically.
2. You use **CDI (Containerized Data Importer)** to upload a Windows Server **.iso** image.
3. You create a **Persistent Volume Claim (PVC)** with **HPP** to store the VM disk locally.
4. You define a **VirtualMachine** resource that uses the uploaded disk.
5. **KVM (Kernel-based Virtual Machine)** runs the VM on the physical host.
6. The **QEMU Agent** inside the VM reports its IP and hostname to the OpenShift Console.
7. You access the VM through a **Virtual Machine Console** using the web console or **RDP** for Windows.
8. If you want to spin up a quick, temporary VM for testing, you can directly create a **VMI (Virtual Machine Instance)**.

| Term | Full Form                    |
|------|------------------------------|
| HCO  | HyperConverged Operator      |
| HPP  | Hostpath Provisioner         |
| QEMU | Quick EMUlator               |
| KVM  | Kernel-based Virtual Machine |
| CDI  | Containerized Data Importer  |
| VMI  | Virtual Machine Instance     |
| RDP  | Remote Desktop Protocol      |

#### Bottom Line

- OpenShift Virtualization uses:
- HCO to manage it all,
- KubeVirt to run VMs in containers,
- KVM + QEMU for the hypervisor,
- CDI & HPP for storage,
- and familiar tools (SSH, RDP, web console) to connect.

One cluster, one toolset, one management plan—run VMs and containers together.

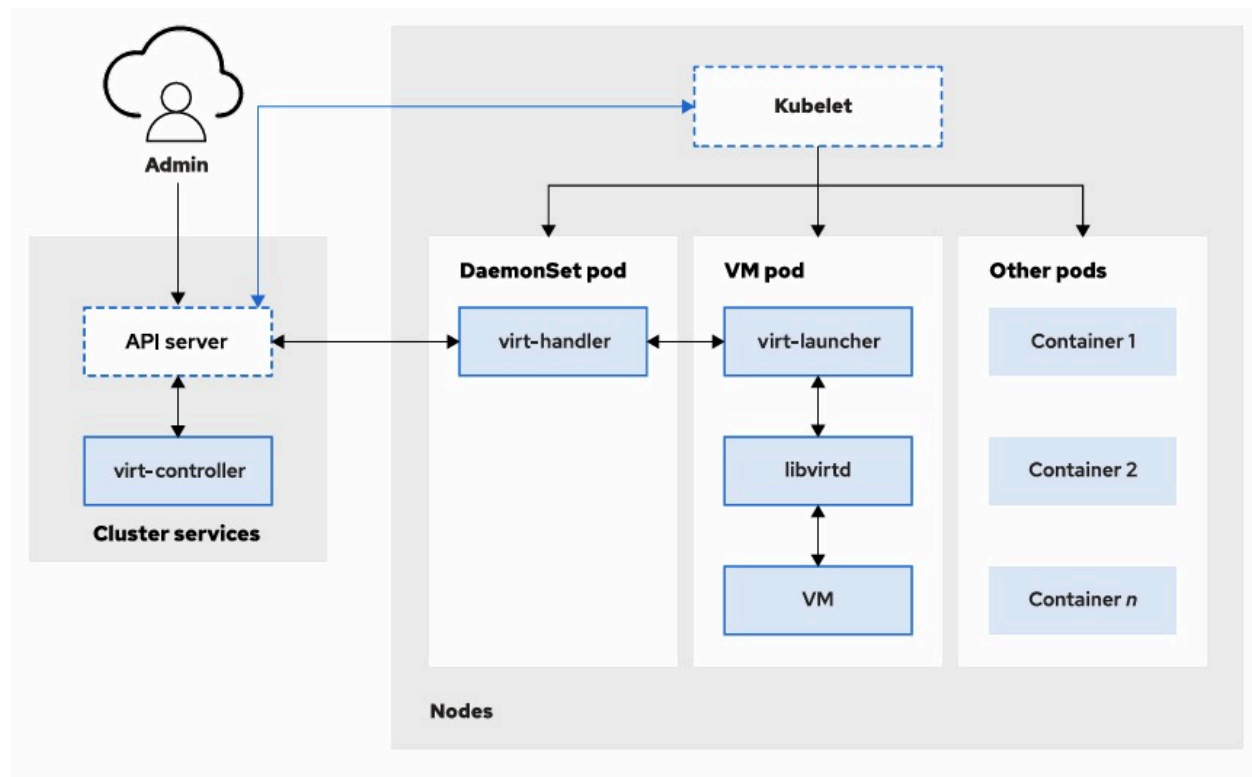
\*\*\*\*\*

## OpenShift Virtualization Components

OpenShift Virtualization supports managing and orchestrating VMs with similar tools and methods as for container management. With these shared tools and methods, administrators and developers can centralize infrastructure management.

### The OpenShift Virtualization Operator

The following diagram shows some of the components of the OpenShift Virtualization operator. *Figure 1.23: OpenShift Virtualization Operator components*



**Figure 1.23: OpenShift Virtualization Operator components**

| Component                                       | Role                                                                           |
|-------------------------------------------------|--------------------------------------------------------------------------------|
| <b>Admin</b>                                    | The user or admin who deploys or manages the VMs.                              |
| <b>API server</b>                               | The Kubernetes API server that receives commands (create VM, delete VM, etc.). |
| <b>virt-controller</b>                          | A controller that watches for VM-related resources and coordinates actions.    |
| <b>virt-handler</b><br>( <i>DaemonSet pod</i> ) | Runs on each node, manages KVM and libvirt for VM lifecycle on that node.      |
| <b>virt-launcher</b> ( <i>VM pod</i> )          | Each VM runs inside its own pod via <b>virt-launcher</b> .                     |
| <b>libvirtd</b>                                 | Manages VM processes on the node.                                              |
| <b>VM</b>                                       | The actual virtual machine instance.                                           |
| <b>Kubelet</b>                                  | The agent on each node that ensures containers and pods run correctly.         |
| <b>Other pods</b>                               | Regular Kubernetes containers run alongside VMs.                               |

### Example: How does it work?

Imagine you want to create a VM named **test-vm** to run a legacy application.

**1. Admin -> API Server:** You (Admin) create a **VirtualMachine** resource using **oc** or the OpenShift Web Console.

Example:

YAML

```
apiVersion: kubevirt.io/v1
```

```
kind: VirtualMachine
```

```
metadata:
```

```
  name: test-vm
```

**2. API Server -> virt-controller:** The request hits the Kubernetes API server. The **virt-controller** notices the new **VirtualMachine** resource and decides how to launch it.

**3. virt-controller -> virt-handler:** The **virt-controller** instructs the **virt-handler** running on the selected node to handle the VM.

**4. virt-handler -> Kubelet:** The **virt-handler** communicates with the **Kubelet** to start a pod (**virt-launcher**) that will host the VM.

**5. virt-launcher pod -> libvirtd -> VM:** Inside the **virt-launcher** pod, the **libvirtd** process starts and manages the VM using **KVM (Kernel Virtual Machine)**. This is how your VM boots up inside a pod, running alongside other regular containers on the node.

**6. Other Pods:** At the same time, the node might run other container pods for applications like web servers or databases — VMs and containers co-exist!

## Key Point

**Benefit:** You can manage VMs just like containers — schedule, monitor, and orchestrate them with Kubernetes-native tools.

So your VM **test-vm** is now a pod, visible with **oc get pods**, and manageable with Kubernetes networking, storage, and security features.

## Summary Example:

You: `oc create -f test-vm.yaml`

-> **API server:** Accepts the request.

-> **virt-controller:** Decides where to run the VM.

-> **virt-handler + Kubelet:** Start `virt-launcher` pod.

-> **virt-launcher:** Runs `libvirtd` to boot the VM. -> **VM:** Your legacy app is now running inside a VM inside a pod.

\*\*\*\*\*

## OpenShift Virtualization Operator Components

The following components of the OpenShift Virtualization operator are described in the image:

- **virt-api:** A cluster-level component that provides an HTTP RESTful entry point for managing VM and VMI-related workflows within the cluster. It updates virtualization Custom Resource Definitions (CRD) and handles defaulting and validating Virtual Machine Instance (VMI) CRDs.
- **virt-controller:** This is a cluster-level component that handles cluster-wide virtualization functions and manages the lifecycle of pods associated with VMIs. The operator uses this component to create the pod where the VM objects execute.
- **virt-handler:** A host-level DaemonSet resource that runs on each node. It monitors for changes to a VM object and performs the necessary operations to meet the desired state.
- **virt-launcher:** The primary container in a VMI-associated pod. The `virt-launcher` component provides control groups (cgroups) and namespaces for hosting the VMI process. The `virt-handler` passes the VM CRD object to the `virt-launcher` component. This component then

uses a container-local **libvirt** instance to start and monitor the VMI process until it terminates.

- **libvirt**: Every VMI pod contains a **libvirt** instance for the **virt-launcher** component to manage the lifecycle of a VMI process.

## Components Explained

### virt-api

- This is the **REST API entry point** for virtualization in OpenShift.
- When you create a VM or VMI, your request hits this API.
- It updates the **Custom Resource Definitions (CRDs)** for Virtual Machines and VMIs.
- **Example:** When you run `oc apply -f test-vm.yaml`, your YAML is processed through **virt-api**. It stores your VM as a Kubernetes CRD.

### virt-controller

- This watches the cluster for **VM/VMI resources**.
- It decides **when** and **where** to run the VM.
- It creates the pod to host the VM process.
- **Example:** Once your **test-vm** CRD is created, **virt-controller** schedules it on a node with available resources.

### virt-handler

- Runs on every node (DaemonSet).
- It talks to the node's **Kubelet** and **virt-launcher**.
- It ensures the VM's pod state matches the desired state.

Here is the text from the image, without emojis:

Example: Your **test-vm** runs inside its own **virt-launcher** pod. This is how the VM gets its dedicated compute resources, networking, and isolation.

### libvirt

- A hypervisor manager inside the **virt-launcher** pod.
- It uses **KVM/QEMU** under the hood to boot the VM.
- It keeps the VM running until you stop or delete it.

**Example:** **libvirtd** inside **virt-launcher** boots your Cirros image, assigns virtual disks, CPU, and memory — just like it would on bare metal.

---

## Putting it all together with a flow

### Flow: Create a VM (**test-vm**)

1. **oc apply -f** -> Request hits **virt-api**.
  2. **virt-controller** sees a new VM CRD -> decides where to run.
  3. **virt-handler** on that node -> creates the **virt-launcher** pod.
  4. **virt-launcher** -> starts **libvirtd** inside the pod.
  5. **libvirtd** boots the VM -> VM runs as a normal Linux process inside the pod's container runtime.
  6. When you stop the VM, **virt-launcher** shuts it down and the pod stops.
- 

## Why is this powerful?

- You manage **VMs with the same Kubernetes tools as containers**.
- VMs get full Kubernetes scheduling, networking, and storage.
- You can run **containers and VMs side by side** — useful for legacy apps and modern workloads together!

\*\*\*\*\*

## The HyperConverged Operator

A required component of OpenShift Virtualization is the **HyperConverged Operator (HCO)**, which includes the following resources

| Resource name                                               | Description                                                                                                       |
|-------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------|
| <code>deployment/hco-webhook</code>                         | Validates the HyperConverged custom resource contents.                                                            |
| <code>deployment/hyperconverged-cluster-cli-download</code> | Provides the <b>virtctl</b> tool binaries to the cluster so that you can download them directly from the cluster. |
| <code>kubevirt/kubevirt-kubevirt-hyperconverged</code>      | Contains all needed operators, CRs, and objects for OpenShift Virtualization.                                     |
| <code>ssp/ssp-kubevirt-hyperconverged</code>                | A Scheduling, Scale, and Performance (SSP) CR.                                                                    |
| <code>cdi/cdi-kubevirt-hyperconverged</code>                | A Containerized Data Importer (CDI) CR.                                                                           |
| <code>networkaddonsconfig/cluster</code>                    | A CR that instructs and is managed by the Cluster Network Addons operator.                                        |

## What is the HyperConverged Operator (HCO)?

The HyperConverged Operator (HCO) is a special operator in **OpenShift Virtualization** that combines all the necessary operators and custom resources (CRs) into a single, unified deployment. Instead of manually installing multiple components, you install the HCO, and it automatically configures everything needed to run VMs within Kubernetes. Think of the HCO as the "**master operator**" for your virtualization environment.

| Resource name                       | Purpose (explained)                                                                                    |
|-------------------------------------|--------------------------------------------------------------------------------------------------------|
| <code>deployment/hco-webhook</code> | Validates your custom resources (like VM or VMI) against schema/rules. Rejects misconfigured VM specs. |



|                                                             |                                                                                                                                              |
|-------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------|
| <code>deployment/hyperconverged-cluster-cli-download</code> | Provides the <b>virtctl</b> tool for admins/users to download. <b>virtctl</b> is used for VM actions (start, stop, console, etc.).           |
| <code>kubevirt/kubevirt-kubevirt-hyperconverged</code>      | The <b>main CR</b> that bundles all Operators for virtualization (KubeVirt, CDI, SSP, Network Addons). It's the <b>brain</b> of HCO.         |
| <code>ssp/ssp-kubevirt-hyperconverged</code>                | <b>SSP (Scheduling, Scale, Performance)</b> : ensures VMs are scheduled and sized properly (CPU/memory templates).                           |
| <code>cdi/cdi-kubevirt-hyperconverged</code>                | <b>CDI (Containerized Data Importer)</b> : automates importing/uploading VM disk images (from ISO, PVC, etc.) into cluster storage.          |
| <code>networkaddonsconfig/cluster</code>                    | Defines <b>network features</b> (multus, SR-IOV, Macvlan, etc.) used by VMs for advanced networking. Managed by the Network Addons Operator. |

### Example: Putting it all together

Let's say you want to create a RHEL VM from an ISO image with advanced networking.

**1. HCO:** You install the HCO — it deploys all these CRs for you.

**2. CDI:** You upload your RHEL ISO using CDI.

```
oc create -f pvc-rhel-iso.yaml
```

**3. SSP:** You use SSP-provided templates for vCPU/memory tuning.

**4. NetworkAddonsConfig:** You configure Multus or SR-IOV for VM NICs.

**5. virtctl:** You download virtctl from

`hyperconverged-cluster-cli-download:`

```
oc get virtclusterversion
virtctl console my-rhel-vm
```

6. **hco-webhook**: Validates your VM spec on creation to ensure no mistakes.

**Result:**

Your RHEL VM boots with the right resources, networking, storage — fully managed by the unified HCO stack.

**Key takeaway**

The HyperConverged Operator bundles:

- VM management (KubeVirt)
- Disk importing (CDI)
- Performance profiles (SSP)
- Advanced networking (Network Addons)
- Validation & tools (hco-webhook, virtctl)

into one supported package for seamless VM orchestration on OpenShift.

\*\*\*\*\*

**Deployed operators**

The default settings include the following operator CRs:

**Containerized Data Importer (CDI)**

The CDI operator components manage the authorization to upload VM disks into PVCs. The CDI also directs external disk upload traffic to the appropriate upload

server pod so that it can be written to the correct PVC and provides a helper pod that imports a virtual machine image into a PVC when creating a data volume.

### Hostpath Provisioner (HPP)

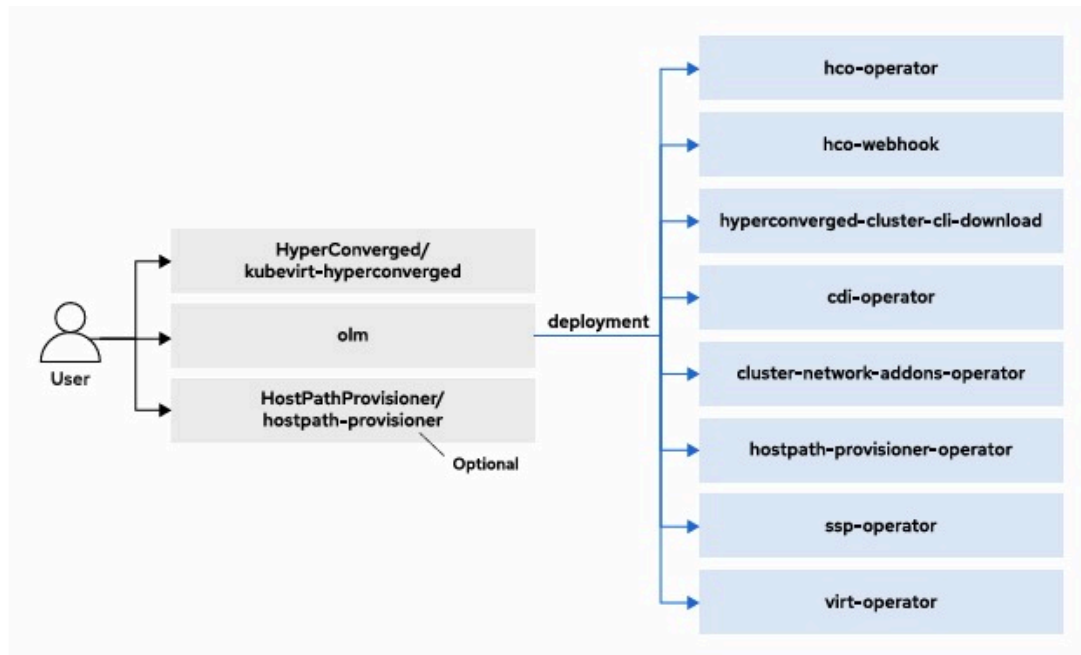
In an environment without available network storage, you can deploy the hostpath provisioner to provision VMs on a node's local storage. The HPP operator includes a deployment resource to provide a worker pod for each node where the HPP is designated to run. The pods mount the backing storage on the node. The operator also deploys two daemons with driver interfaces of the HPP.

### Cluster Network Addons

The Cluster Network Addons operator deploys networking components on a cluster and manages the related resources for extended network functions.

### Scheduling, Scale, and Performance (SSP)

The SSP operator deploys common templates, related default boot sources, pipeline tasks, and the template validator.



## What happens when you deploy the HyperConverged Operator (HCO)?

When you deploy the **HyperConverged Operator (HCO)**, you create a single custom resource (CR) which triggers the automatic installation and management of several other operators required for OpenShift Virtualization.

Specifically, the **kubevirt-hyperconverged** CR deploys the following operators:

- **hco-operator**
- **cdi-operator**
- **ssp-operator**
- **virt-operator**
- **cluster-network-addons-operator**
- *(Optionally)* **hostpath-provisioner-operator**

This process simplifies deployment; instead of manually installing and configuring multiple operators, you just deploy one master CR and the HCO handles the rest.

## Key Operators Explained

### Containerized Data Importer (CDI)

The **Containerized Data Importer (CDI)** manages the process of importing VM disk images. It handles the traffic from your local machine to the OpenShift storage and deploys helper pods to perform the import of images such as ISO or PVC files. For example, if you want to run a Windows VM, you would upload a Windows ISO file, and CDI would store it as a PVC, which your VM can then use to boot from.

### Hostpath Provisioner (HPP) — Optional

- **What it does:**
  - Provides local storage on each worker node.

- It is useful for **labs, dev/test**, or environments without external storage.
- It deploys a **pod per node** to mount local disk paths for VM disks.
- It also deploys daemons with driver interfaces to manage storage paths.
- **Example:**
  - If you don't have NFS, Gluster, or Ceph, you can use HPP to make your node's **/var/data** directory a VM disk storage backend.

## Cluster Network Addons

- **What it does:**
  - Adds advanced networking for VMs.
  - It deploys and manages **Multus, SR-IOV, Macvlan**, and other extended network functions.
- **Example:**
  - You can use Cluster Network Addons to enable your VM to have multiple network interface cards (NICs) on separate VLANs.

## Scheduling, Scale, and Performance (SSP)

- **What it does:**
  - Deploys common **VM templates** (e.g., RHEL, CentOS).
  - Manages **boot sources** for these templates.
  - Provides **pipeline tasks** for building images.
  - Includes a **template validator** to ensure new VM templates meet policy/standards.
- **Example:**
  - When you create a VM from the OpenShift console, the ready-made templates are provided by SSP. It ensures your VMs use the correct size for CPU/memory, boot disk, etc.

## Bringing it all together

When you apply the `kubevirt-hyperconverged.yaml` file, **OpenShift Virtualization** deploys all the necessary components:

- **VM runtime** (`virt-operator`)
- **Disk importer** (`cdi-operator`)
- **Local storage backend** (`hostpath-operator` - if enabled)
- **Advanced networking** (`cluster-network-addons-operator`)
- **Templates & performance tuning** (`ssp-operator`)
- **A webhook for resource validation** (`hco-webhook`)
- **A CLI helper** (`virtctl` via `hyperconverged-cluster-cli-download`)

All these components are interconnected, allowing you to manage your VMs like containers — scheduled, networked, and stored by Kubernetes-native tools.

## Key Benefit

Single deployment (HCO) -> full virtualization stack -> zero manual wiring -> run VMs next to containers -> unified Kubernetes lifecycle!

If you'd like, I can:

- Provide an **end-to-end YAML sample** with HostPath storage.
- Show **commands to upload a disk with CDI**.
- Or create a **step-by-step mini lab** (upload ISO -> create PVC -> create VM -> boot).

\*\*\*\*\*

# Multus Secondary Networks

## Objectives

- Expose applications to external access by using a secondary network.

## Using Different Networks

Kubernetes manages a pod network and a service network. The **pod network** provides a network interface to each pod and handles communication between all pods. The **service network** provides stable addressing for services running on pods. Additional features exist to expose services outside the cluster.

In certain situations, connecting some pods to a different network can be beneficial. For instance, a dedicated network with dedicated resources can enhance the performance of specific traffic. Furthermore, a dedicated network can have different security properties from the default network, helping to meet security requirements. Using extra interfaces can also simplify tasks, such as managing outgoing traffic from pods.

The **Multus CNI (container network interface)** plug-in helps attach pods to custom networks. These custom networks can be either external networks already in use or custom networks created for internal cluster use.

## Summary from the Image

### Objective

- Expose applications to **external access** using a secondary network.

### Key Concepts

- Kubernetes has:
  - **Pod Network:** for communication between pods.
  - **Service Network:** for stable service discovery and access.

- Sometimes, using a **different network (secondary)** for some pods is necessary for:
  - **Performance improvements**
  - **Security segregation**
  - **Traffic control**
- **Multus CNI:** A plug-in that allows attaching pods to **multiple networks**.
- These networks may be:
  - **External to the cluster** (e.g., physical network)
  - **Internal custom networks**
- **Query successful**

## Use Case Example: Network Segregation for Database Access

### Scenario

You are running a web application (frontend) and a database (backend) in a Kubernetes cluster. You want:

- The frontend to be publicly accessible over the internet.
  - The database to be accessible only over a secure, isolated network (not reachable from the internet or other pods unnecessarily).
- 

### Without Multus (default behavior)

- All pods (frontend and database) are on the same default pod network.
  - Any pod can communicate with the database pod unless NetworkPolicies are set up.
  - External access to the database must go through a Kubernetes service or Ingress.
- 

### With Multus Secondary Network (Better Approach)



1. **Primary network:** All pods still have access to the default Kubernetes pod network.
2. **Secondary network (via Multus):**
  - A separate interface is added to the database pod.
  - This secondary interface is connected to an isolated VLAN or physical NIC that is accessible only by trusted components (e.g., DB monitoring tools or backup servers).

## Benefits

- **Improved security:** The database is on an isolated network.
  - **Performance:** Dedicated network bandwidth for database traffic.
  - **Compliance:** Follows network isolation policies.
- 

## YAML Example

Here is a basic example of how a pod would look when using Multus to connect to a secondary network:

```
apiVersion: v1
kind: Pod
metadata:
  name: db-pod
  annotations:
    k8s.v1.cni.cncf.io/networks: secondary-net
spec:
  containers:
    - name: database
      image: postgres:16
      ports:
        - containerPort: 5432
```

NetworkAttachmentDefinition Example for **secondary-net**:

```

apiVersion: "k8s.cni.cncf.io/v1"
kind: NetworkAttachmentDefinition
metadata:
  name: secondary-net
spec:
  config: '{
    "cniVersion": "0.3.1",
    "type": "macvlan",
    "master": "eth1",
    "ipam": {
      "type": "static",
      "addresses": [
        {
          "address": "192.168.10.10/24",
          "gateway": "192.168.10.1"
        }
      ]
    }
  }'

```

**eth1** is the host NIC that connects to the secondary physical network.

This pod now has **two interfaces**:

- Default interface via CNI (for internal pod communication)
- **eth1** via Multus for external/private DB access

| Feature                           | Without Multus               | With Multus Secondary Network |
|-----------------------------------|------------------------------|-------------------------------|
| <b>Network interfaces</b>         | 1                            | 2 or more                     |
| <b>Security Isolation</b>         | Limited                      | High (via separate networks)  |
| <b>External access control</b>    | Needs services/load balancer | Custom control per interface  |
| <b>Network performance tuning</b> | Not granular                 | Per network, per pod          |

## Overview

The diagram compares **three virtualization approaches** in Red Hat technologies:

1. **Red Hat Virtualization (RHV)**
2. **OpenShift Virtualization**
3. **Red Hat OpenStack Platform (OSP)**

All three use **libvirt + QEMU/KVM** at their core, but differ in orchestration and management layers.

